UNIVERSIDADE FEDERAL DO CEARÁ

Relatório Final

Disciplina: Qualidade de Software
Equipe: Pedro Rivaldo Tabosa Silva - 511881
Projeto: LightRAG - https://github.com/HKUDS/LightRAG
Fork: https://github.com/pedrorivald/LightRAG - branch refatoracao-test-smells (refatorações + métricas) e branch test/refactor-test-smells (usada no PR)
Pull Request: https://github.com/HKUDS/LightRAG/pull/4290
Commit SHA: 453dce83d6d0354a06e46c8d4029a0895c4e054b

1. Apresentação do projeto

O LightRAG é um framework open source em Python, mantido pelo HKU Data Intelligence Lab, para Retrieval-Augmented Generation (RAG) baseado em grafos de conhecimento. Ele extrai entidades e relações dos documentos inseridos, monta um grafo de conhecimento e o combina com busca vetorial para responder perguntas em diferentes modos de recuperação (local, global, hybrid, naive e mix). Além da biblioteca, o projeto oferece um servidor FastAPI com interface web e suporte a vários backends de armazenamento (JSON, NetworkX, PostgreSQL, Neo4j, Redis, Milvus, Qdrant etc.) e provedores de LLM. Serve a desenvolvedores e pesquisadores que querem montar sistemas de perguntas e respostas sobre suas próprias bases de documentos.

Os testes usam pytest e ficam na pasta tests/, organizada espelhando o pacote lightrag/ (tests/api, tests/chunker, tests/kg/<backend>_impl, tests/llm/<provedor>_impl, tests/parser, tests/pipeline etc.). No commit base, o TEMPY analisou 537 arquivos de teste e o pytest coletou 9.142 testes. Os testes são marcados como offline (rodam sem serviços externos e são o gate do CI) ou integration (precisam de banco de dados ou API e são pulados por padrão). A suíte faz uso intenso de fixtures, monkeypatch, asyncio e dublês de teste escritos à mão.

2. Estado inicial do projeto

2.1 Test smells detectados antes da refatoração (TEMPY)

Test smell	Ocorrências detectadas
Conditional Test Logic	749
Verbose Test	328
Exception Handling	327

Test smell	Ocorrências detectadas
Unknown Test	324
Programming Paradigms Blend	264
Redundant Print	142
Undefined Test	39
Sleepy Test	22
Non-Functional Statement	16
Total	2.211

2.2 Resultados dos testes e cobertura antes da refatoração (Pytest)

Preencha com base no arquivo metrics-before-pytest/pytest_antes.html e no relatório de cobertura.

As 7 falhas não estão relacionadas ao código de teste: 5 testes de SVG precisam da biblioteca de sistema cairo, ausente no meu macOS, e 2 testes de tests/api/test_path_prefixes.py recebem um redirecionamento 307 no lugar do 200 esperado no meu ambiente. Os 246 ignorados são, em sua maioria, testes de integração (pulados por padrão) e testes condicionados à plataforma ou à versão do Python.

Métrica	Valor
Testes aprovados (Passed)	8.889
Testes com falha (Failed)	7
Erros (Errors)	0
Ignorados (Skipped)	246
Cobertura total (%)	76,09% (linhas + branches)

2.3 Métricas do código de teste antes da refatoração (Radon)

Use apenas as linhas com origem = teste dos CSVs da pasta metrics-before-radon/.

Métrica	Valor
SLOC total dos testes (linha TOTAL com origem = teste)	157.827
Complexidade ciclomática média das funções de teste	2,898 (12.805 funções)

Métrica	Valor
Índice de manutenibilidade médio dos arquivos de teste	56,83 (599 arquivos)

O Radon não conseguiu analisar o arquivo tests/parser/docx/test_smart_heading_features.py (SyntaxError na linha 541), que ficou registrado em arquivos_com_erro_antes.csv e fora das médias.

3. Refatorações realizadas

3.1 Seleção das ocorrências

Explique quais tipos de test smell você escolheu e por quê, considerando a severidade indicada no CATSPY.

Escolhi 4 tipos e refatorei 25 ocorrências: Exception Handling (6), Conditional Test Logic (9), Unknown Test (6) e Non-Functional Statement (4).

Três deles têm severidade alta no CATSPY (Exception Handling, Conditional Test Logic e Unknown Test) e são, também, os três tipos mais frequentes no projeto entre os de severidade alta que dava para refatorar com segurança. Os outros tipos de severidade alta ou média ficaram de fora por esses motivos:

- Verbose Test (alta): resolver exige quebrar testes de 45+ linhas em fixtures e auxiliares, uma mudança grande e arriscada num projeto que eu não conhecia, e difícil de revisar num PR.
- Undefined Test (alta): as ocorrências são funções auxiliares com assert (ex.: main() e _check_*), que não devem virar testes coletados pelo pytest; seriam mudanças sem ganho real.
- Sleepy Test (média): os sleeps estão em testes de concorrência, onde a espera faz parte do cenário (corrida de eventos). Removê-los muda o que o teste verifica.
- Programming Paradigms Blend (média): mudança estrutural no módulo inteiro (misturar classes e funções), sem benefício claro para o projeto.

Como quarto tipo, fiquei com Non-Functional Statement (severidade baixa). O Redundant Print, também de severidade baixa, foi descartado porque seus 142 casos estão em 2 arquivos de teste de integração que precisam de um servidor rodando e foram pulados na execução, então eu não teria como provar que continuariam passando.

Dentro de cada tipo, só selecionei ocorrências genuínas, porque o TEMPY gera muitos falsos positivos: das 327 ocorrências de Exception Handling, 304 eram blocos try/finally de limpeza (finalize_storages); das 324 de Unknown Test, 245 eram testes que verificam com pytest.raises, que o TEMPY não reconhece como asserção. Também priorizei testes offline que já passavam no estado inicial.

3.2 Lista das refatorações

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R01	Exception Handling	tests/test_create_prefixed_exception.py::test_jsondecodeerror_does_not_raise_and_preserves_message	c58772c39	Sim	Sim	Não
R02	Exception Handling	tests/kg/json_impl/test_atomic_write_write_json.py::test_write_json_concurrent_writers_land_intact	0cd442e1b	Sim	Sim	Não
R03	Exception Handling	tests/kg/networkx_impl/test_networkx_atomic_write.py::test_write_nx_graph_concurrent_writers_no_race	c7032eaaf	Sim	Sim	Não
R04	Exception Handling	tests/pipeline/test_pipeline_release_closure.py::test_resume_skips_purge_when_chunks_list_empty	4f70745f0	Sim	Sim	Não
R05	Exception Handling	tests/llm/test_tokenizer_contract.py::test_builtin_tokenizer_survives_concurrent_encode_and_decode	2bc95285a	Sim	Sim	Não
R06	Exception Handling	tests/pipeline/test_tolerant_load_json_dict.py::test_greedy_regex_would_fail_same_input	ba9ae3415	Sim	Sim	Não
R07	Conditional Test Logic	tests/utils/test_normalize_rerank_result.py::test_bool_index_is_rejected_even_though_bool_is_an_int_subclass	7375f2149	Sim	Sim	Não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R08	Conditional Test Logic	tests/api/routes/test_document_routes_docx_archive.py::test_sanitize_filename_rejects_separator_and_control_characters	6584c191c	Sim	Sim	Não
R09	Conditional Test Logic	tests/api/routes/test_document_routes_docx_archive.py::test_sanitize_filename_rejects_colon	7dabede00	Sim	Sim	Não
R10	Conditional Test Logic	tests/api/routes/test_document_routes_docx_archive.py::test_sanitize_filename_allows_other_windows_reserved_characters	5d214e0a6	Sim	Sim	Não
R11	Conditional Test Logic	tests/pipeline/test_chunk_options_regex_scrub.py::test_empty_addon_params_falls_back_to_env_default	b21d91666	Sim	Sim	Não
R12	Conditional Test Logic	tests/parser/external/test_common.py::test_env_bool_truthy_falsy (1º laço → test_env_bool_truthy)	b155b63bb	Sim	Sim	Não
R13	Conditional Test Logic	tests/parser/external/test_common.py::test_env_bool_truthy_falsy (2º laço → test_env_bool_falsy)	2536721da	Sim	Sim	Não
R14	Conditional Test Logic	tests/pipeline/test_multimodal_path_containment.py::test_non_string_path_value_is_refused_not_raised	35afc774c	Sim	Sim	Não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R15	Conditional Test Logic	tests/kg/test_reservation_primitives.py::test_reservation_owner_kind_reads_a_full_owner_record	383ac6ed3	Sim	Sim	Não
R16	Unknown Test	tests/parser/external/test_common.py::test_clear_dir_contents_noop_when_missing	d95170f15	Sim	Sim	Não
R17	Unknown Test	tests/kg/test_bounded_pipeline_history.py::test_a_missing_history_is_skipped_not_raised	0dd044a8b	Sim	Sim	Não
R18	Unknown Test	tests/kg/test_bounded_pipeline_history.py::test_a_failing_extend_is_swallowed	5f289f55c	Sim	Sim	Não
R19	Unknown Test	tests/kg/test_pipeline_status_logger.py::test_never_raises_even_when_diagnostic_logging_itself_fails	310a96d13	Sim	Sim	Não
R20	Unknown Test	tests/parser/docx/test_zip_budget.py::test_validate_source_does_not_open_the_archive	e1bb5d1b7	Sim	Sim	Não
R21	Unknown Test	tests/parser/docx/test_zip_budget.py::test_ordinary_docx_passes_untouched	df989c2b7	Sim	Sim	Não
R22	Non-Functional Statement	tests/kg/test_scheduling_backend_fixture.py::test_isolation_leaves_an_unset_variable_unset	cdc915b71	Sim	Sim	Não
R23	Non-Functional Statement	tests/llm/gemini_impl/test_gemini_llm.py::tes	24feebf30	Sim	Sim	Não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
		t_gemini_rejects_typed_response_format				
R24	Non-Functional Statement	tests/api/test_custom_chunker_startup.py::startup (fixture)	032079db6	Sim	Sim	Não
R25	Non-Functional Statement	tests/pipeline/test_pipeline_release_closure.py::test_resume_purges_old_chunks_when_content_already_extracted	d247c44ad	Sim	Sim	Não

Além das 25 ocorrências selecionadas, as refatorações R02, R03 e R05 removeram de quebra 9 ocorrências de Conditional Test Logic (laços de start/join de threads e os if dentro do worker), e a R04 removeu 1 Non-Functional Statement (o pass do except). Cada refatoração foi validada com pytest no arquivo, ruff format/check e o TEMPY no modo --arquivo. Para conferir se surgiu algum smell novo, comparei os pares (tipo, método) do smells.csv antes e depois em todos os 18 arquivos alterados: nenhum apareceu.

3.3 Exemplos detalhados

R01 – Exception Handling (tests/test_create_prefixed_exception.py::test_jsondecodeerror_does_not_raise_and_preserves_message)

Antes:

index.tsx

```python
@pytest.mark.offline
def test_jsondecodeerror_does_not_raise_and_preserves_message():
    try:
        json.loads('{\n  "x": "abc')  # unterminated string → JSONDecodeError
    except json.JSONDecodeError as exc:
        result = create_prefixed_exception(exc, "`entity`")
    # JSONDecodeError(msg) is missing (doc, pos): falls back to a clean RuntimeError
    assert isinstance(result, RuntimeError)
    assert result.args  # never an empty/garbled exception
    assert "`entity`" in str(result)
    assert "JSONDecodeError" in str(result)
    assert "Unterminated string" in str(result)
```

Depois:

index.tsx

```python
@pytest.mark.offline
def test_jsondecodeerror_does_not_raise_and_preserves_message():
    with pytest.raises(json.JSONDecodeError) as exc_info:
        json.loads('{\n  "x": "abc')  # unterminated string → JSONDecodeError
    result = create_prefixed_exception(exc_info.value, "`entity`")
    # JSONDecodeError(msg) is missing (doc, pos): falls back to a clean RuntimeError
    assert isinstance(result, RuntimeError)
    assert result.args  # never an empty/garbled exception
    assert "`entity`" in str(result)
    assert "JSONDecodeError" in str(result)
    assert "Unterminated string" in str(result)
```

Por que era um test smell: o teste usava try/except só para capturar a exceção que depois inspeciona, um tratamento manual que o framework já oferece. Pior: se json.loads deixasse de lançar a exceção, o teste quebraria com um NameError (a variável result nunca seria definida), uma falha confusa que não diz o que deu errado.

Estratégia do CATSPY aplicada: a do catálogo, usar o recurso de exceções do framework. O exemplo do CATSPY é em unittest (assertRaises); adaptei para o equivalente do pytest, with pytest.raises(...) as exc_info, usando exc_info.value no restante do teste.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	7	6
SLOC do arquivo de teste (raw_por_arquivo_e_total)	28	27

R06 – Exception Handling (tests/pipeline/test_tolerant_load_json_dict.py::test_greedy_regex_would_fail_same_input)

Antes:

index.tsx

```python
def test_greedy_regex_would_fail_same_input() -> None:
    """The old greedy ``\\{.*\\}`` slice over-extends across trailing braces and
    drops the object; the new helper recovers it."""
    import json_repair
    import re

    raw = '{"facts":[{"text":"ok"}]} trailing {brace}'
    m = re.search(r"\{[\s\S]*\}", raw)
    assert m is not None
    try:
        obj = json_repair.loads(m.group(0))
        recovered = obj if isinstance(obj, dict) else {}
    except Exception:
        recovered = {}
    assert recovered == {}
    assert tolerant_load_json_dict(raw) == {"facts": [{"text": "ok"}]}
```

Depois:

index.tsx

```python
def _legacy_greedy_load(raw: str) -> dict:
    """The pre-fix parser: greedy ``\\{.*\\}`` slice, then json_repair.

    Returns ``{}`` when the slice does not repair into an object, whether
    json_repair raises or returns a non-dict (depends on its version).
    """
    import json_repair
    import re

    m = re.search(r"\{[\s\S]*\}", raw)
    if m is None:
        return {}
    try:
        obj = json_repair.loads(m.group(0))
    except Exception:
        return {}
    return obj if isinstance(obj, dict) else {}


def test_greedy_regex_would_fail_same_input() -> None:
    """The old greedy ``\\{.*\\}`` slice over-extends across trailing braces and
    drops the object; the new helper recovers it."""
    raw = '{"facts":[{"text":"ok"}]} trailing {brace}'
    assert _legacy_greedy_load(raw) == {}
    assert tolerant_load_json_dict(raw) == {"facts": [{"text": "ok"}]}
```

Por que era um test smell: havia um try/except Exception no corpo do teste, misturando a reprodução do parser antigo com as asserções e escondendo a intenção do teste.

Estratégia do CATSPY aplicada: a estratégia do catálogo (pytest.raises) não se aplicava: ao executar, vi que o json_repair não lança exceção para essa entrada (devolve uma lista), e o try existe só para tolerar versões da biblioteca que lancem. Apliquei Extract Method: a lógica do parser antigo foi para a função auxiliar _legacy_greedy_load, e o teste ficou com duas asserções que dizem exatamente o que ele afirma. Não deixei um assert dentro da função auxiliar porque o TEMPY passaria a acusar um Undefined Test.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	6	3 (+ 4 na função auxiliar)
SLOC do arquivo de teste (raw_por_arquivo_e_total)	154	156

R12/R13 – Conditional Test Logic (tests/parser/external/test_common.py::test_env_bool_truthy_falsy)

Antes:

index.tsx

```
def test_env_bool_truthy_falsy(monkeypatch: pytest.MonkeyPatch) → None:
    for raw in ("1", "true", "yes", "ON"):
        monkeypatch.setenv("X", raw)
        assert env_bool("X", False) is True
    for raw in ("0", "false", "no", "off"):
        monkeypatch.setenv("X", raw)
        assert env_bool("X", True) is False
```

Depois:

index.tsx

```
@pytest.mark.parametrize("raw", ["1", "true", "yes", "ON"])
def test_env_bool_truthy(monkeypatch: pytest.MonkeyPatch, raw: str) → None:
    monkeypatch.setenv("X", raw)
    assert env_bool("X", False) is True


@pytest.mark.parametrize("raw", ["0", "false", "no", "off"])
def test_env_bool_falsy(monkeypatch: pytest.MonkeyPatch, raw: str) → None:
    monkeypatch.setenv("X", raw)
    assert env_bool("X", True) is False
```

Por que era um test smell: o teste tinha dois laços for. Se o primeiro valor falhasse, os demais nem eram verificados, e o relatório não dizia qual valor quebrou. Além disso, um único teste cobria dois comportamentos diferentes (valores verdadeiros e falsos).

Estratégia do CATSPY aplicada: o CATSPY sugere separar os caminhos em testes distintos. Separei em test_env_bool_truthy e test_env_bool_falsy (um commit por laço) e, em vez de escrever um teste por valor, usei @pytest.mark.parametrize, que é a forma idiomática no pytest e gera um caso por valor.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	5	2 + 2 (dois testes)
SLOC do arquivo de teste (raw_por_arquivo_e_total)	293	296 (inclui a R16)

R20 – Unknown Test (tests/parser/docx/test_zip_budget.py::test_validate_source_does_not_open_the_archive)

Antes:

index.tsx

```python
def test_validate_source_does_not_open_the_archive(tmp_path, monkeypatch):
    src = _benign(tmp_path / "ok.docx")

    def _explode(*args, **kwargs):  # pragma: no cover - only on regression
        raise AssertionError("validate_source opened the zip on the event loop")

    monkeypatch.setattr(zipfile, "ZipFile", _explode)
    NativeDocxParser().validate_source(src, "ok.docx")
```

Depois:

index.tsx

```python
def test_validate_source_does_not_open_the_archive(tmp_path, monkeypatch):
    src = _benign(tmp_path / "ok.docx")

    opened: list[tuple] = []

    def _explode(*args, **kwargs):  # pragma: no cover - only on regression
        opened.append(args)
        raise RuntimeError("validate_source opened the zip on the event loop")

    monkeypatch.setattr(zipfile, "ZipFile", _explode)
    NativeDocxParser().validate_source(src, "ok.docx")
    # Recorded as well as raised, so a broad except in validate_source cannot
    # swallow the regression.
    assert opened == []
```

Por que era um test smell: o teste não tinha nenhuma asserção: dependia de o ZipFile falso lançar AssertionError. Como AssertionError é subclasse de Exception, um except Exception que alguém viesse a colocar no validate_source engoliria a regressão e o teste continuaria passando.

Estratégia do CATSPY aplicada: a do catálogo, adicionar uma asserção explícita sobre o resultado esperado. O dublê passou a registrar cada abertura numa lista (além de lançar exceção), e o teste verifica opened == []. Verifiquei que hoje não existe esse except no validate_source; a mudança protege contra o futuro.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	1	2
SLOC do arquivo de teste (raw_por_arquivo_e_total)	532	537 (inclui a R21)

R22 – Non-Functional Statement (tests/kg/test_scheduling_backend_fixture.py::test_isolation_leaves_an_unset_variable_unset)

Antes:

index.tsx

```python
def test_isolation_leaves_an_unset_variable_unset(monkeypatch):
    monkeypatch.delenv("REDIS_WORKSPACE", raising=False)
    with _workspace_env_isolated("REDIS_WORKSPACE"):
        pass
    assert "REDIS_WORKSPACE" not in os.environ
```

Depois:

index.tsx

```python
def test_isolation_leaves_an_unset_variable_unset(monkeypatch):
    monkeypatch.delenv("REDIS_WORKSPACE", raising=False)
    with _workspace_env_isolated("REDIS_WORKSPACE"):
        assert "REDIS_WORKSPACE" not in os.environ
    assert "REDIS_WORKSPACE" not in os.environ
```

Por que era um test smell: o corpo do with era só um pass: o teste entrava no isolamento e não verificava nada lá dentro.

Estratégia do CATSPY aplicada: a do catálogo, trocar a instrução sem efeito pela asserção correspondente. Usei a mesma verificação que o teste vizinho faz no mesmo ponto: a variável fica ausente durante o isolamento, além de depois.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	2	3
SLOC do arquivo de teste (raw_por_arquivo_e_total)	40	40

4. Estado final e comparação

Preencha com base nas pastas metrics-before- e metrics-after-*.*

Métrica	Antes	Depois	Variação
Total de test smells (TEMPY)	2.211	2.176	-35
Testes aprovados	8.889	8.916	+27
Testes com falha	7	7	0
Cobertura total (%)	76,09	76,09	0,00 p.p.
SLOC dos testes	157.827	157.828	+1
Complexidade ciclomática média das funções de teste	2,898	2,896	−0,002
Índice de manutenibilidade médio dos arquivos de teste	56,829	56,838	+0,009

Test smell	Antes	Depois	Variação
Conditional Test Logic	749	731	-18
Verbose Test	328	328	0
Exception Handling	327	321	-6
Unknown Test	324	318	−6
Programming Paradigms Blend	264	264	0
Redundant Print	142	142	0
Undefined Test	39	39	0
Sleepy Test	22	22	0
Non-Functional Statement	16	11	-5
Total	2.211	2.176	-35

5. Análise dos resultados

Com base nas tabelas das Seções 2, 3 e 4, responda:

Quais métricas melhoraram e quais pioraram?

Melhoraram: o total de test smells caiu de 2.211 para 2.176 (−35, −1,6%), com quedas nos quatro tipos trabalhados (Conditional Test Logic −18, Exception Handling −6, Unknown Test −6, Non-Functional Statement −5). A complexidade ciclomática média das funções de teste caiu levemente (2,898 → 2,896) e o índice de manutenibilidade médio subiu levemente (56,829 → 56,838). O número de testes aprovados subiu de 8.889 para 8.916 (+27), mas isso não é cobertura nova: é o efeito do parametrize, que transforma cada valor de um laço num caso de teste próprio.

Ficaram estáveis: cobertura (76,09%, idêntica) e falhas (as mesmas 7). O SLOC dos testes ficou praticamente igual (+1): o parametrize acrescenta a linha do decorador, as asserções novas do Unknown Test e a função auxiliar da R06 acrescentam linhas, enquanto a troca das threads manuais por ThreadPoolExecutor (R02, R03, R05) remove linhas.

Piorou, à primeira vista: a complexidade ciclomática de cada função refatorada para Unknown Test e Non-Functional Statement subiu (de 1 para 2–4 nas R16 a R21 e de 2 para 3 na R22). Isso acontece porque o Radon conta cada assert como um ponto de decisão (é um if implícito). Ou seja, ali a métrica aumentou justamente porque o teste passou a verificar algo. Nas refatorações de Exception Handling e Conditional Test Logic, a complexidade de cada função caiu (ex.: R02 de 10 para 6, R05 de 7 para 4, R12/R13 de 5 para 2 + 2).

Em termos absolutos as variações das médias são pequenas porque o projeto é muito grande (12.805 funções de teste em 599 arquivos) e mexi em 18 arquivos.

Alguma refatoração introduziu novos test smells? Quais?

Não. Comparei os pares (tipo de smell, método) do smells.csv antes e depois em todos os 18 arquivos alterados e nenhum smell novo apareceu. Em um caso isso influenciou a solução: na R06, manter o assert m is not None dentro da nova função auxiliar faria o TEMPY acusar um Undefined Test (função com asserção sem o prefixo test_), então reescrevi a lógica sem a asserção. Nas refatorações com threads, também evitei laços e ifs dentro dos workers (usei list comprehension), porque o TEMPY os contaria como Conditional Test Logic.

O resultado dos testes ou a cobertura mudou?

A cobertura não mudou (76,09%), o que era esperado: só alterei código de teste, e os mesmos caminhos do código-fonte continuam sendo exercitados. As 7 falhas são exatamente as mesmas antes e depois e não têm relação com as refatorações (biblioteca cairo ausente para os testes de SVG e um redirecionamento 307 em test_path_prefixes). Os 246 testes ignorados também são os mesmos. Todos os testes que passavam antes continuaram passando.

O que mudou foi a quantidade de testes coletados: 9.142 → 9.169 (+27), por causa do parametrize (por exemplo, o test_env_bool_truthy_falsy virou 8 casos, 4 para cada teste). O tempo total da suíte variou de 315 s para 264 s, mas isso depende da carga da máquina e não deve ser atribuído às refatorações.

Quais tipos de smell foram mais fáceis e mais difíceis de refatorar?

Mais fácil: Conditional Test Logic. Nos casos que escolhi, todos laços for sobre listas fixas, a troca por @pytest.mark.parametrize é mecânica e o ganho é claro (cada valor vira um caso e a falha aponta o valor exato). Non-Functional Statement também foi simples de executar, mas foi difícil de justificar, porque 3 dos 4 casos eram o pass no corpo de classes dublê.

Mais difícil: Exception Handling. A maioria das ocorrências era falso positivo (try/finally de limpeza), e as genuínas exigiam entender o contexto: exceções dentro de threads (R02, R03, R05), um except: pass que só podia ser removido depois de confirmar que o pipeline não lança exceção nesse cenário (R04, confirmado simulando o defeito) e um caso em que a estratégia do catálogo não se aplicava (R06). Unknown Test foi intermediário: adicionar uma asserção é fácil, mas encontrar uma que não fosse trivial (algo além de assert f() is None) exigiu ler a implementação de cada função testada.

6. Uso do CATSPY

Como você usou o CATSPY durante a atividade?

Antes de refatorar cada tipo, consultei a página do smell no CATSPY. Usei a definição para decidir se cada ocorrência apontada pelo TEMPY era genuína, já que muitas não eram, e o exemplo refatorado como ponto de partida para a solução. Também usei a severidade para priorizar os tipos (seção 3.1) e os efeitos negativos para justificar as mudanças nas mensagens de commit e na descrição do PR.

Quais partes do catálogo mais ajudaram (definição, efeitos negativos, exemplo com problema, exemplo refatorado)?

A definição e o par exemplo com problema / exemplo refatorado foram as partes mais úteis: deixam claro o que é o smell e qual é a direção da correção. Os efeitos negativos ajudaram a argumentar por que a mudança vale a pena (por exemplo, "pode ofuscar a intenção do teste e reduz a confiabilidade" no Exception Handling). Os exemplos de contexto (ciência de dados, web) ajudaram menos no meu caso.

Houve casos em que a estratégia do catálogo não se aplicava diretamente ao projeto? Como você resolveu?

Sim, em alguns casos:

- Os exemplos do catálogo são em unittest e o LightRAG usa pytest. Precisei traduzir: assertRaises virou pytest.raises, e "um método de teste para cada caso" virou @pytest.mark.parametrize.
- R06: a sugestão para Exception Handling (usar o recurso de exceções do framework) não se aplicava, porque o código não lança exceção para aquela entrada. Resolvi com Extract Method para uma função auxiliar.
- R02, R03 e R05: as exceções acontecem dentro de threads, onde o pytest.raises não ajuda. Resolvi com ThreadPoolExecutor.map, que relança no teste a exceção do worker.
- R23 a R25: o catálogo trata o pass como placeholder de teste não implementado, mas aqui ele era o corpo obrigatório de classes dublê. Resolvi trocando o pass por uma docstring que explica o papel da classe.

O que faltou ou poderia ser melhorado no catálogo?

- Exemplos em pytest (pytest.raises, parametrize, fixtures), já que é o framework mais usado em Python.
- Orientação sobre falsos positivos e sobre quando o smell é aceitável. Por exemplo, try/finally de limpeza não é Exception Handling, e testes com pytest.raises não são Unknown Test, embora o TEMPY marque os dois.
- Casos de testes concorrentes (exceções em threads) e de Non-Functional Statement fora de método de teste (corpo de classes dublê).

7. Dificuldades e lições aprendidas

Quais foram as principais dificuldades (ambiente, dependências, entendimento do código, ferramentas, revisão do PR)?

Ambiente: O LightRAG não tem o extra [dev] indicado no tutorial, então instalei os mesmos extras do CI (api, offline-storage, offline-llm e pytest), e o radon, o pytest-html e o pytest-cov à parte. Também descobri que um novo uv sync remove esses pacotes extras.

Execução: Sete testes falham no macOS por motivos de ambiente, e precisei confirmar que não tinham relação com o meu trabalho.

Entendimento do código: o projeto é grande e cheio de regras de domínio (concorrência do pipeline, contratos de armazenamento). Para escrever asserções com sentido nos Unknown Test e para remover o except da R04, precisei ler a implementação das funções testadas.

Ferramentas: o TEMPY usa heurísticas por linha (qualquer try:, qualquer for no início da linha), o que gera muitos falsos positivos. Filtrar as ocorrências genuínas tomou boa parte do tempo.

PR: a branch main do LightRAG avançou 14 commits desde o commit base. Por isso, criei para o PR uma branch separada a partir do main atual e apliquei nela os 25 commits com cherry-pick, sem nenhum conflito. Antes de enviar, rodei de novo os testes dos arquivos alterados e os hooks de pre-commit do projeto, seguindo as regras de contribuição (ruff, mensagens de commit em inglês).

O que você aprendeu com a atividade?

Aprendi que ferramentas de detecção de test smells são um ponto de partida e não um veredito: cada ocorrência precisa de julgamento humano. Também aprendi que um teste que "só não pode lançar exceção" quase sempre pode afirmar algo melhor, e que vale provar que um teste continua capaz de falhar, simulando o defeito, como fiz na R04. Sobre métricas, ficou claro que um número pior nem sempre significa código pior: a complexidade subiu nos testes que ganharam asserções. Por fim, refatorar em passos pequenos, um commit por ocorrência e validando cada um, facilitou muito a revisão e a rastreabilidade.

8. Contribuição (Pull Request)

Descreva o que foi enviado no PR e se houve alguma resposta dos mantenedores.

Abri o Pull Request #4290 (https://github.com/HKUDS/LightRAG/pull/4290) para o repositório original (HKUDS/LightRAG, branch main), a partir da branch test/refactor-test-smells do meu fork. Como o main tinha avançado 14 commits desde o commit base da disciplina, criei essa branch a partir do main atual e apliquei os 25 commits de refatoração (um por ocorrência), sem conflitos. O PR altera apenas 18 arquivos de teste da pasta tests/ (+130/−114 linhas) e nenhum código de produção. Antes de abri-lo, confirmei que os 472 testes dos arquivos alterados passam e que todos os hooks de pre-commit do projeto (ruff format, ruff check, trailing whitespace, end-of-file) estão ok.

A descrição segue o template do projeto (Description, Related Issues, Changes Made, Checklist, Additional Notes). Nela expliquei o objetivo, agrupei as mudanças por tipo de smell, informei que só alterei testes e contei que algumas detecções da ferramenta eram falsos positivos e foram deixadas de lado. As pastas de métricas e os scripts de extração não fazem parte do PR: depois de abri-lo, fiz o commit deles na branch refatoracao-test-smells do fork (commit 2f7a5f5e7), como pede a atividade.

Até o momento, não houve resposta dos mantenedores.

9. Observações gerais

Use este espaço para registrar o que não coube nas seções anteriores: problemas com o ambiente ou com as ferramentas, decisões tomadas durante a atividade, sugestões para a ferramenta etc.

- Falsos positivos do TEMPY: 304 das 327 ocorrências de Exception Handling eram try/finally de limpeza, e 245 das 324 de Unknown Test eram testes com pytest.raises. Os números que o TEMPY reporta exageram o problema real do projeto nesses dois tipos.
- Redundant Print não foi trabalhado porque está concentrado em testes de integração que não rodam sem um servidor (pulados na medição).
- Além das 25 ocorrências selecionadas, 10 foram removidas de quebra (9 Conditional Test Logic e 1 Non-Functional Statement), o que explica a queda total de 35 smells.
- O Radon não conseguiu analisar tests/parser/docx/test_smart_heading_features.py (SyntaxError na linha 541) nas duas medições; o arquivo ficou fora das médias.
- O registro detalhado de cada refatoração (smell, dificuldade, uso do CATSPY e método) está no Diário de Bordo.